Riebeckite

Markdownを書く。
サイトは、自分で組む。

ObsidianノートとMarkdownを、TypeScriptとHonoXで
自由に組み立てる静的サイトフレームワーク。

CONTENT → SITE BUILD

01 **Write**

content/notes/*.md
Obsidian Vault

02 **Resolve**

parse · links · permalink
Plugin hooks

03 **Build**

static HTML + islands
HonoX / Vite

YOUR CONTENT. YOUR CODE.

v0.0.32 Apache-2.0 riebeckite.dev

BACKGROUND

出発点は、Quartz 4への不満だった

Markdownを書けばサイトになる。その基本はよくできていた。問題は、その先にあった。

クライアントUI	HooksとDOM操作。普段使っているコンポーネントの書き方ではなかった。
構成の分離	Vaultとサイトのコードを分けて管理するのが難しい。
アップグレード	v4からv5へ移るとき、サイトのセットアップをやり直すことになった。

> Markdownは入力であって、アーキテクチャの中心ではありません。
>
> README_ja — Riebeckiteを選ぶ理由

だから、生成物の前提を変えた。

設定済みの製品ではなく、普通のHonoXアプリケーションを出力する。

WHAT IT IS

Markdownを、自分のコードで組むサイトへ

Riebeckiteは、ObsidianノートやMarkdownを静的サイトとして公開するためのフレームワークです。生成されるのは、@riebeckite/* を依存に持つ普通のHonoXアプリケーションです。

STACK

いつもの道具

TypeScript、JSX、HonoX、npm。独自のテンプレート言語を覚える必要はない。

CONTENT

コンテンツは手元に

フォルダでも、既存のVaultでも、別リポジトリでも。content.directoryで指定する。

ECOSYSTEM

npmで拡張する

PluginもThemeも普通のnpmパッケージ。公式に72のPluginと6のThemeがある。

OWNERSHIP　サイトはフレームワークの付属物ではない。更新は依存パッケージの更新で済み、自分のタイミングでバージョンを上げられる。

PRINCIPLES

四つの設計原則

フレームワークの形を決めているのは、機能の数ではなく境界の引き方です。

01

内側は外側を知らない

CoreはHonoX、Vite、Cloudflare、特定のPluginを知らない。

02

Markdownは入力

ページ、URL、リンク、アセットはビルド時に解決する。Markdownはアーキテクチャの中心ではない。

03

静的を基準にする

JavaScriptがなくても本文が読める。動きは必要な場所だけに足す。

04

permalinkが正

ファイルパスからURLを再計算しない。解決済みのpermalinkだけを公開URLとして使う。

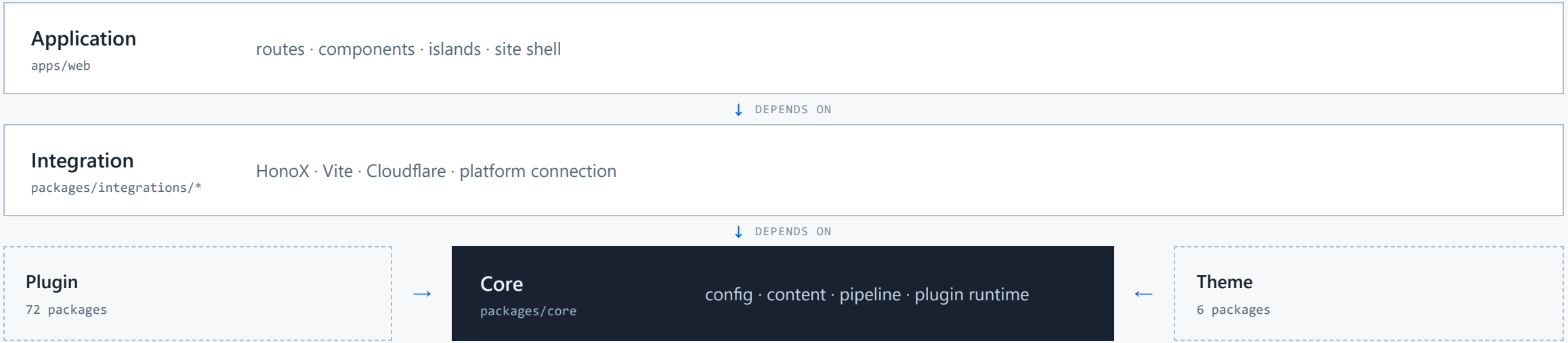
迷ったら、その機能を成立させるために必要な、最も内側のpackageに置く。

ARCHITECTURE

依存は、常に内側へ

ApplicationはIntegrationへ、Integration・Plugin・ThemeはCoreへ依存する。

CLIはIntegrationとCoreを使う。Coreから外側への依存は作らない。



境界の例 – 「PLUGINのページをHONOXで表示したい」

CORE	PLUGIN	INTEGRATION	APP
framework非依存のPage contract	Pageを提供する	Pageをrouteに接続する	最終的なdocumentを描画する

CONTENT SYSTEM

コンテンツが、公開URLになるまで

ContentSourceが「どこから読むか」、ContentManagerが「どう扱うか」を担当する。

ContentSource

scan · read · hash

ファイル、Vault、別リポジトリ。保存場所を抽象化する。

どこから読むか

→

ContentManager

location · parse · hooks

公開URLを解決し、Pluginを通してコンテンツを処理する。

どう扱うか

→

Manifest / Graph

pages · links · assets

サイト全体の事実をひとつのmanifestにまとめる。

何が公開されるか

→

HonoX build

routes · islands

静的HTMLと、必要なブラウザコードを出力する。

どう届けるか

ファイルパス

`content/posts/hello.md`

→

論理パス

`posts/hello`

→

公開URL

`/blog/hello/`

SOURCE OF TRUTH

ファイルパスからURLを推測しない。**Pluginやconfigが解決したpermalink**だけを使う。

PLUGIN ECOSYSTEM

拡張は、npmパッケージで

PluginはMarkdownの変換だけでなく、ページ、コンポーネント、CSS、ブラウザコード、HTTPエンドポイント、生成ファイルまで追加できる。

領域	公式PLUGINの例
Markdown	obsidian-markdown · properties · citations · shortcodes
図表	mermaid · plantuml · d2 · graphviz · wavedrom · chartjs
発見	search · backlinks · local-graph · toc · query · bases
執筆	code-enhance · diff · flashcards · kanban · marp
公開	seo · webmention · share · l10n · deploy · analytics

72
OFFICIAL PLUGINS

6
OFFICIAL THEMES

Themeは見た目だけを担当する。route、Page Type、application componentは持たない。

Pluginは機能、Themeは見た目。どちらもCoreが公開しているcontractだけを使う。

PLUGIN API

Pluginは、ひとつの関数から

definePlugin()に、必要な拡張ポイントだけを渡す。サイト内のファイルでも、npmパッケージでも同じ仕組み。

extensions/local-plugin.ts — TypeScript

```ts
import { definePlugin } from "@riebeckite/core";

export function localPlugin() {
  return definePlugin({
    name: "local",
    assets: [
      {
        pluginName: "local",
        kind: "style",
        moduleSpecifier: "/extensions/plugin.css",
      },
    ],
  });
}
```

変換	remarkPlugins / rehypePlugins / extendMarkdownPipeline
コンテンツ	onContentLoaded / onPostParsed / resolveContentLocations
表示	renderers / pageTypes / assets / clientEntries
配信	endpoints / seo / addDiagnostics
依存	provides / requires / order

> **nameだけでもPluginになる。**必要になった拡張ポイントを、ひとつずつ足していく。

配布しないなら、`extensions/local-plugin.ts` に置くだけでよい。packageにするのは、他のサイトでも使うときだけ。

BUILD

ビルドは、変更だけをやり直す

前回の結果を使い、変更された部分だけを処理する。stateが使えなければ、最初からやり直せばいい。

01 **設定とPluginを読む**
configを解決し、依存関係を検証する。

02 **コンテンツを読み、変換する**
Plugin hooksを通してMarkdownを処理する。

03 **公開URL・リンク・アセットを確定する**
サイト全体のmanifestとcontent graphを作る。

04 **HonoXでbuildする**
静的HTMLと、必要なislandsを生成する。

Incremental build

`.riebeckite/build/content-state.json`

- 変更されたコンテンツだけを再利用する
- 安全に再利用できるか判断できなければfull build
- buildが失敗したら、前回の成功状態を保持する
- `--full` で最初から処理する

Build-time と Runtime

BUILD-TIME
CLI · Doctor · Inspector · cache · profile

RUNTIME
生成済みアプリと安定したcontent dataだけ。Workersはstateを読まない。

高速化のためのstateがなくても、同じサイトを正しく生成できる。これを前提にしている。

SPA NAVIGATION

SPAは、静的な土台を壊さない

サーバー描画を基準に、ブラウザ側だけを強化する。置き換えるのはページのルートで、HTML文書全体ではない。



01 **クリックを横取りし、次のHTMLを取得する**
通常のリンク遷移を、fetchに置き換える。

02 **ページスコープをunmountし、headを更新する**
後始末をしてから、メタデータを差し替える。

03 **mainを置き換え、islandsをhydrateする**
シェルは残したまま、ページのルートだけを更新する。

04 **履歴、スクロール、フォーカスを更新する**
最後にページスコープをmountし、navigationイベントを発火する。

`spa: { prefetch: "hover", viewTransitions: true }` で、先読みと画面遷移を有効にできる。

WORKFLOW

作る、確かめる、届ける

雛形の生成からデプロイまで、CLIで完結する。

terminal — quick start

```
# 雛形を生成
$ npx create-riebeckite

# 依存を入れて、開発サーバーを起動
$ cd my-site
$ npm install
$ npm exec riebeckite dev

# 静的サイトを dist/ へ
$ npm exec riebeckite build
```

dev	開発サーバーを起動する
check	設定とPluginを検証する
doctor	環境、content、output、Themeを診断する（--fix）
inspect	config、plugins、content、graph、buildの解決結果を見る
build	静的サイトを生成する（--fullで全処理）
deploy	Cloudflare Workersへ公開する

プリセットは `starter` / `showcase` / `minimal` / `empty`。`--list-presets` で一覧できる。

REPOSITORY

責務ごとに、packageを分ける

pnpm workspaceのmonorepo。内側のpackageは、外側の実装を知らない。

```
riebeckite/
├ packages/
│  ├ core/              config · content · pipeline · plugin runtime
│  ├ cli/               create · dev · check · build · deploy
│  ├ integrations/      honox · analytics · webmention
│  ├ plugins/           72 official plugins
│  ├ themes/            default · minimal · gruvbox · ...
│  └ create-riebeckite/ site generator
├ apps/web/            公式サイト / 参照アプリ
├ docs/                ドキュメントの原本
├ templates/           デプロイ用テンプレート
└ riebeckite.config.ts サイト設定
```

apps/webも、参照アプリのひとつ。

Core、Plugin、Theme、Integrationが正しく組み合わさることを確認する。公式ドキュメントサイトを兼ねている。

v0.0.32	72	6	3	Apache-2.0
VERSION	PLUGINS	THEMES	INTEGRATIONS	LICENSE

FIT

向いている人、向かない人

Riebeckiteが向くのは、サイトを自分のプロダクトとして持ちたい人。

向いている

- ObsidianやMarkdownで書いている
- サイトのUIも自分で作りたい
- TypeScript / JSX / HonoXを普段使っている
- コンテンツをサイトの外に置きたい

向かない

- コードを書かずにサイトを完成させたい
- 既製テーマを選ぶだけで十分
- Markdownの表示だけを最速で公開したい

> **表示だけなら、ホスティング型のサービスで足りる。**サイトの構造まで自分で決めたいなら、ここから始める。

Your content.
Your code.
Your framework.

Obsidianで書き、TypeScriptで組み、自分のタイミングで公開する。

Documentation Plugin catalog Showcase GitHub

CREATE A SITE

```
npx create-riebeckite
```

starter / showcase
minimal / empty

Publish your knowledge. Build without limits.